```
var handleCors = function(options) {
  return function(req, res, next) {
    …
    if (options.allowCredentials) {
      res.set('Access-Control-Allow-Credentials', 'true');
    }
    …
};
```

Reads allowCredentials option to set Access-Control-Allow-Credentials header

The code introduces a new `allowCredentials` option. Setting this value to `true` means that cookies are allowed on requests.

If you allow credentials on requests, the Access-Control-Allow-Origin header must be set to an actual origin value. The `*` value isn't valid when using credentials, and the browser will reject any response that has Access-Control-Allow-Credentials set to `true` but Access-Control-Allow-Origin set to `*`.

Using similar code as the previous section, you could write code to always echo the Origin header value when `allowCredentials` is set to `true`. This would allow any client (internal or external) to include cookies on requests. But while this is technically possible, it's not advisable. By enabling cookies to be always included on requests, you open your server up to a class of vulnerabilities called cross-site request forgery (CSRF). Appendix C goes into more details on CSRF. The rest of this section explores how to protect against CSRF vulnerabilities.

VALIDATING THE ORIGIN

The simplest way to protect against CSRF is to return an error if the origin is invalid. This ensures that only valid origins are allowed, and that the request stops processing if the origin is invalid. The previous section showed you how to use a whitelist to validate the origin, but it continued to process the request even when it encountered an invalid origin. The following listing repeats the origin validation code from the previous section.

Listing 6.6 Only whitelisted origins can make CORS requests

```
var originWhitelist = [
  'null',
  'http://localhost:1111'
];

var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true
};

var handleCors = function(options) {
  return function(req, res, next) {

    if (options.allowOrigin) {
      var origin = req.headers['origin'];
      if (options.allowOrigin(origin)) {
        res.set('Access-Control-Allow-Origin', origin);
      }
```

Nothing happens if origin is invalid, and request continues processing.